1. Setup and Agent Configuration

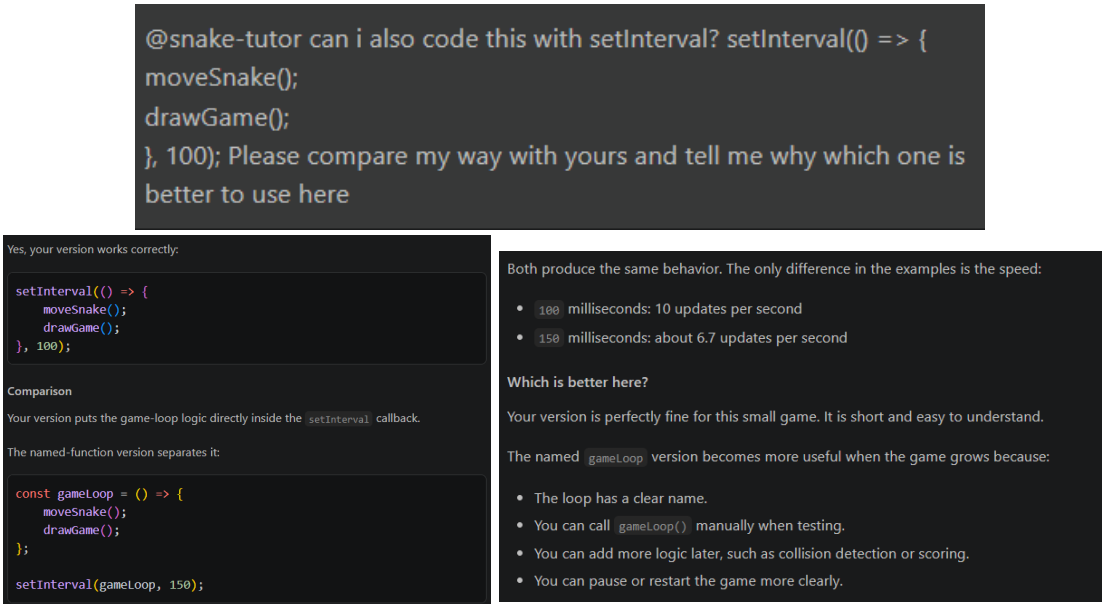
Since I already had an active GitHub student account, setting up Copilot wasn't necessary. However, creating a custom .agent.md file and using Copilot Chat as an interactive programming tutor was a completely new experience. I created the .github/agents/ folder and configured snake-tutor.agent.md according to my preferred learning style. Key changes included:

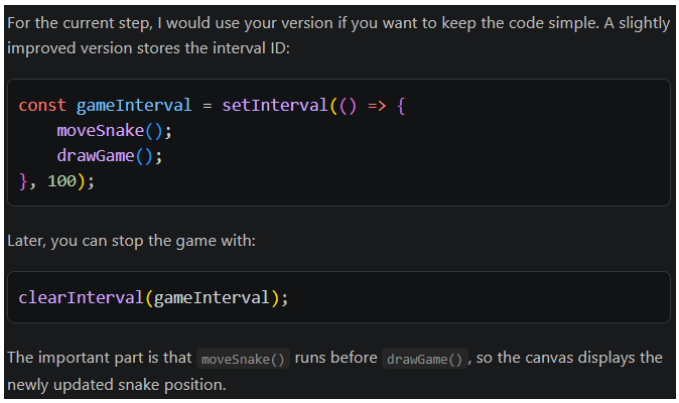
- Teaching style: I prioritized a direct, motivating tone and focused on conceptual explanations rather than immediate solutions.
- Detailed explanations: The agent should explicitly explain JavaScript concepts, keywords, and functions (e.g., setInterval, addEventListener, array methods) before presenting code examples.
- Structured sub-steps: Large requirements were broken down into smaller milestones with helpful hints.
- Version control and best practices: I added explicit reminders to commit changes after each step, suggested formatted commit messages, recommended Git branching for complex functions, and encouraged code commenting.

My general workflow involved defining my goals, independently trying out implementations, questioning given advice, and reviewing suggestions before integration.

2. Development process

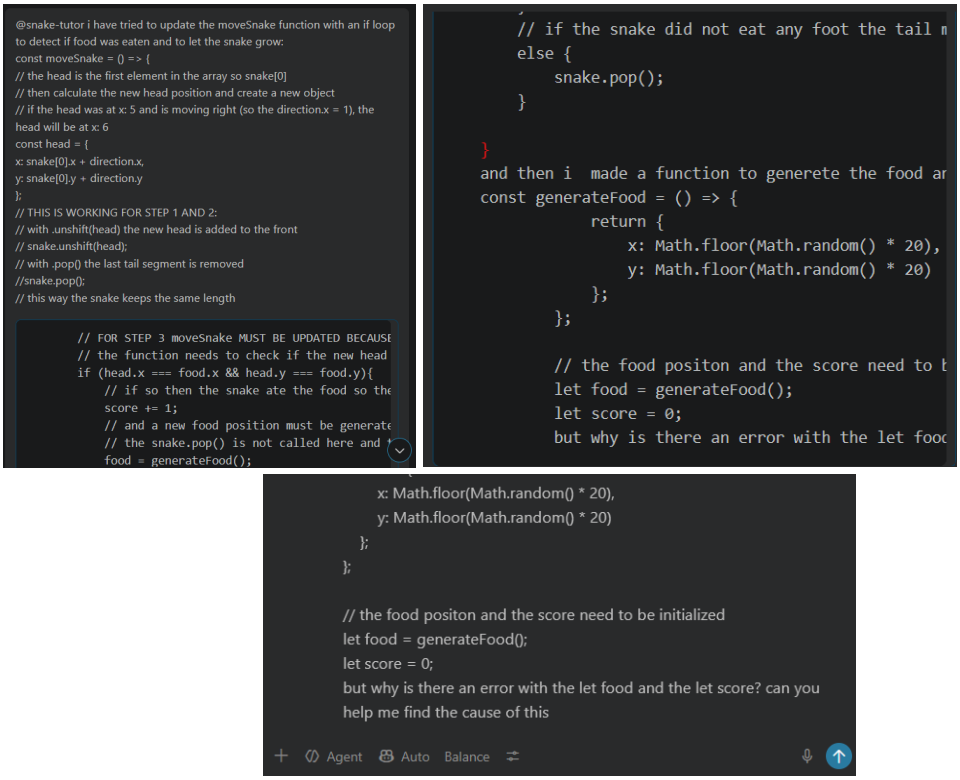
Moment 1: Choosing the Game-Loop Structure

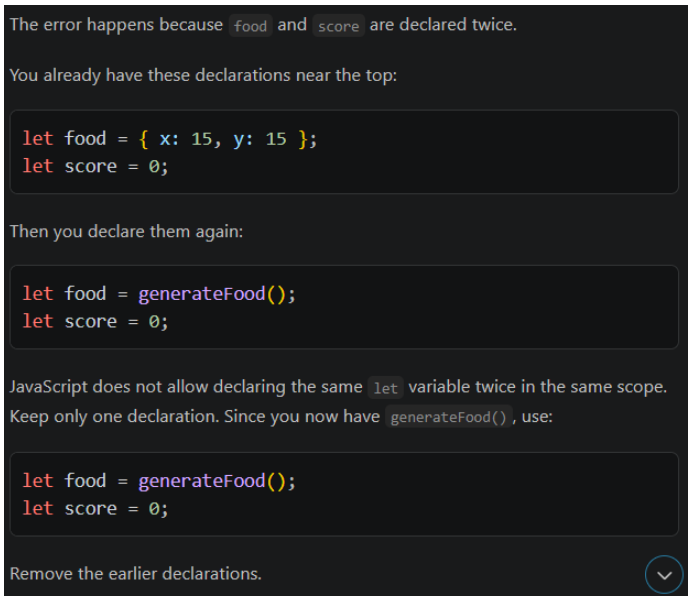




When I started working on the movement logic, I considered passing an anonymous function directly to “setInterval()”. However, the tutor suggested defining a separate function, “gameLoop()”. I asked him to compare both approaches instead of simply adopting his version. He explained that both would work, but a named function would be easier to manage as the game's complexity increased. I chose his approach because I wanted to expand my game and avoid complications later on.

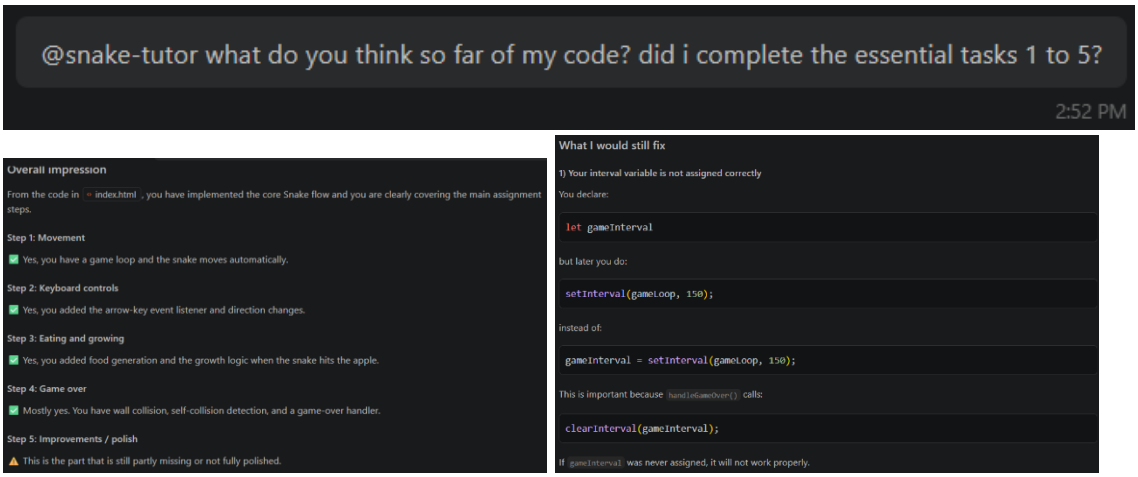
Moment 2: Debugging the pause button

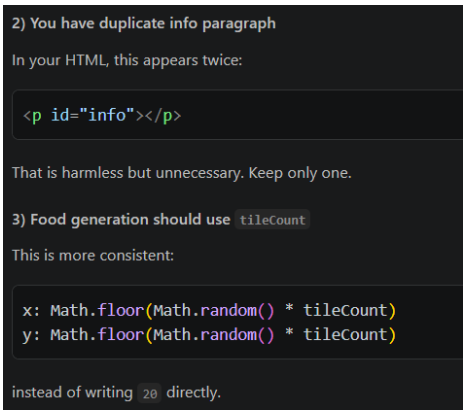


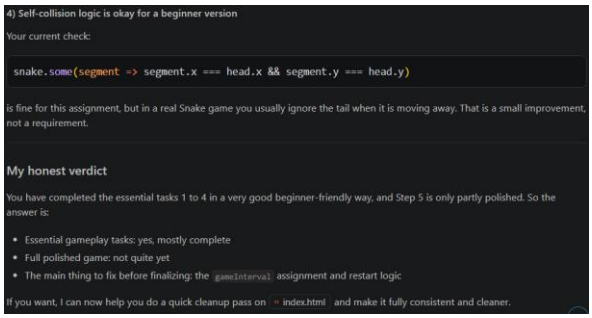


When implementing the play/pause function, pressing the spacebar twice triggered the pause state. I initially assumed that removing the focus (blur()) was necessary, but the tutor disagreed and explained that the browser's default key event forwarding was activating the focused key. I learned about “event.preventDefault()”, which resolved the issue and was helpful later too.
I really appreciated that the tutor didn't just agree with me, as other AI tools might have done, but instead questioned my reasoning. I also learned that my tutor has access to all my code files and that I don't need to provide him with the code every time. This became clear to me because he was referencing elements and lines that I hadn't shown him.

Moment 3: Reviewing my progress

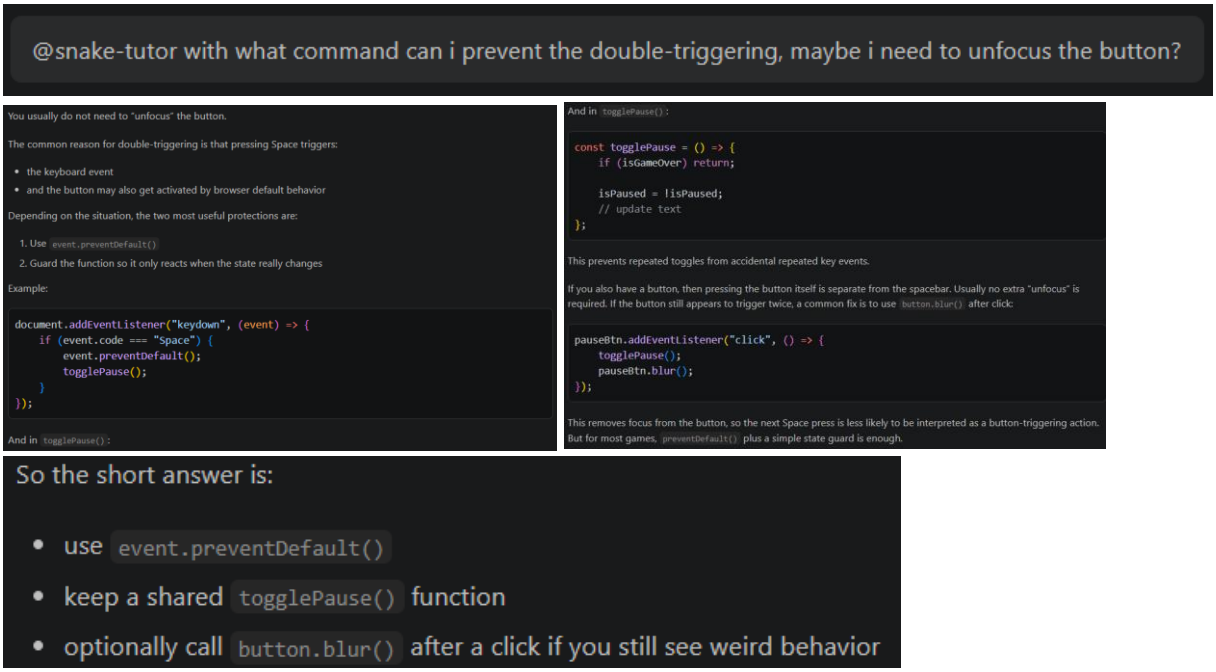






I asked the tutor to check if I had completed all the necessary steps. Instead of giving me the expected general approval or rejection, he evaluated my functional implementations, highlighted potential edge cases, and suggested minor refinements without my prompting. This way, however, I was able to fix minor errors before they escalated into larger problems.

Moment 4: Being corrected instead of simply agreeing



When I wanted to fix a bug, I suggested a solution to the tutor. Instead of simply agreeing with me, he explained that it wasn't correct and pointed me to another one. So, the tutor didn't just blindly agree with me. I had to rethink my reasoning and understand why my solution wouldn't work before changing the code.

Moment 5: Git branch and merge



To experiment with my code, I created a branch that I could merge back into after making changes. However, I wasn't sure if this had worked because the Git history looked different than expected. Instead of a list of instructions, the agent asked for permission to run diagnostic commands directly in my VS Code terminal. He identified a fast-forward merge and explained how it worked. I found this moment very impressive, as the tutor could execute commands directly in my local terminal and asked for my permission beforehand. This transparency shows that it's not a black-box model, but rather a "human-in-the-loop" control system. Later, when I asked the

tutor to do a final check to see if my project fulfilled all requirements before submission, it went even further: it requested access to my running game environment. After I granted permission, the agent autonomously played through and tested the game, verifying button handlers, movement, obstacles, portals, wall collisions, high score persistence, pausing and saving/loading. Seeing the AI move from passive code review to actively testing my live application was incredible.

3. Reflection

My modifications resulted in Copilot acting more like a programming instructor than a passive code generator. By reducing code outputs and adding mandatory explanations, I was able to write more code myself and understand why a solution worked. Compared to a more general AI programming tool, I didn't have to repeatedly ask it to explain every piece of code or withhold the finished solution.

I used many JavaScript concepts from the lectures, such as “event.preventDefault()”, “pop()”, event listeners, DOM manipulation, and variable scope. I learned about unfamiliar concepts, like “setInterval()”, “clearInterval()”, and audio functions, through the tutor and online research.

There were clear differences between the chat tutor and the inline suggestions. While the tutor had access to my entire project (existing variable names, functions, HTML structure, etc.) and his suggestions were relevant and logical, the inline suggestions were completely out of context and often used incorrect variable names. I used them primarily just for inspiration.

I didn't notice any mistakes from the tutor. I always questioned his answers and tried to understand them. The only thing that went wrong was that he forgot to remind me to commit. Next time, I would formulate instructions more precisely and perhaps include that I first have to try to explain an error before it's resolved, in order to improve my debugging skills.

The standard Copilot primarily functions as an assistant, but the customized tutor is similar to an AI agent in an AI-powered IDE. It has access to my workspace and can execute commands there, but its responses remain in a chat and not directly in the document. This gives you more control over your project while still providing more contextual and automated support.

Git-Hub Repository:

https://github.com/J-u-lia/AI_Assignment